CS5494 Assignment 1 - Task1.3

- Name: Li Shuo
- EID: sli986

MNIST Handwritten Digit Generation with a Convolutional GAN

In this experiment, under the same budget of 20 training iterations, we compared the baseline model with a model using a lower learning rate and conducted experiments to isolate the effects of BatchNorm on the generator.

The baseline achieved **89.5% human recognizability** on 200 fixed evaluation samples, compared with **84.0%** for the lower learning rate and **82.0%** without generator BatchNorm. All three runs covered ten recognizable digit classes, but their class distributions were uneven. These findings describe one training seed and one human annotation pass.

In [ ]:
```
from report_utils import load_report_data, show_table, show_figure, show_checkpo
report = load_report_data()
```

1. Objective and experimental questions

The generator learns to transform Gaussian noise into a digit image, while the discriminator learns to distinguish training images from generated images. The experiment targets:

1. Whether the baseline produces recognizable and varied digits.
2. How reducing both learning rates affects results within a fixed budget.
3. How removing generator BatchNorm changes learning and final samples.

E0 is a preliminary debugging run. We use E1-E3 to compare.

2. Data preparation and loading

The input is **original grayscale MNIST**, with image shape $1 \times 28 \times 28$. 60,000-image training split;No binary values; No augmented.

For an original pixel $u \in \{0, \ldots, 255\}$, preprocessing is

$$x = 2\left(\frac{u}{255}\right) - 1,$$

giving a `float32` tensor in $[-1, 1]$. This range matches the generator's final Tanh activation. For display, pixels are mapped back using $(x + 1)/2$.

prepare_data.py acquires the raw files, adds the channel dimension, performs the transformation, and saves the training tensor. mnist_data.py loads it and returns **images only**. MNIST digit labels are used for data-distribution statistics, not as inputs to G or D, because the model is unconditional.

The loader shuffles each epoch and keeps the final incomplete batch: 469 batches with a final batch of 96 images for the full set, or 47 batches with a final batch of 112 for E0. Its random stream is separate from model initialization and training-noise sampling.

In [ ]:
```
show_figure(report, 'outputs/data/training_samples.png', width=560)
```

Figure 1. The first 64 real training images in their original order, displayed in [0,1].

3. Model modules and adversarial learning

3.1 Generator

The generator G receives $z \sim \mathcal{N}(0, I_{100})$. A fully connected layer creates a small spatial feature representation, and two transposed convolutions learn successive upsampling operations. The implementation is `Generator` in models.py. Here, B denotes the actual batch size.

Stage	Operation	Output shape	Function
Input	Gaussian noise	$B \times 100$	Supplies variation between generated samples
Projection	Linear(100, 6272), ReLU	$B \times 6272$	Learns an initial nonlinear feature representation
Reshape	Unflatten	$B \times 128 \times 7 \times 7$	Organizes the features spatially
Upsampling 1	ConvTranspose2d(128, 64), kernel 4, stride 2, padding 1	$B \times 64 \times 14 \times 14$	Learns spatial upsampling and feature mixing
Normalization	BatchNorm2d(64), ReLU	$B \times 64 \times 14 \times 14$	Normalizes intermediate channels and applies a nonlinearity

Stage	Operation	Output shape	Function
Upsampling 2	ConvTranspose2d(64, 1), kernel 4, stride 2, padding 1	$B \times 1 \times 28 \times 28$	Produces the grayscale image
Output	Tanh	$B \times 1 \times 28 \times 28$	Restricts generated pixels to $[-1, 1]$

The baseline generator has **765,761 trainable parameters**. E3 replaces the single BatchNorm layer with Identity, leaving the other layers unchanged. Its generator has **765,633 parameters**, a reduction of 128 channel-scale and bias parameters. BatchNorm running statistics are saved buffers, rather than trainable parameters.

3.2 Discriminator

The discriminator D extracts local image features, reduces spatial resolution, and outputs one **logit** per image. Both real and generated images use the same input shape and range.

Stage	Operation	Output shape	Function
Input	Grayscale image	$B \times 1 \times 28 \times 28$	Accepts a real image or $G(z)$
Feature block 1	Conv2d(1, 64), kernel 4, stride 2, padding 1; LeakyReLU(0.2)	$B \times 64 \times 14 \times 14$	Extracts local features and downsamples
Feature block 2	Conv2d(64, 128), kernel 4, stride 2, padding 1; LeakyReLU(0.2)	$B \times 128 \times 7 \times 7$	Builds a deeper image representation
Flatten	Flatten all non-batch dimensions	$B \times 6272$	Prepares features for the output layer
Classifier	Linear(6272, 1)	$B \times 1$	Outputs a real-versus-generated logit

1. LeakyReLU keeps a nonzero slope for negative inputs. D has **138,561 trainable parameters** and contains no BatchNorm or Dropout.
2. No final Sigmoid because `BCEWithLogitsLoss` incorporates that operation.

3.3 Losses and alternating updates

Let $\ell(s, t)$ denote binary cross-entropy applied to a logit s and target t through `BCEWithLogitsLoss`. The objectives are

$$L_D = \mathbb{E}_x \ell(D(x), 1) + \mathbb{E}_z \ell(D(G(z)), 0),$$

$$L_G = \mathbb{E}_{z'} \ell(D(G(z')), 1).$$

The two terms in D's loss are each averaged over their batch and then added, without dividing their sum by two. G uses the non-saturating objective, which encourages D to classify generated images as real.

train.py performs the following operations for every batch:

1. Generate fake images, evaluate real images against target 1 and detached fake images against target 0, and take one Adam step for D. Detaching prevents this loss from propagating into G.
2. Resample noise, freeze D's parameters, evaluate the new generated images against target 1, and take one Adam step for G. The computation through D remains differentiable with respect to its input.
3. Restore D's trainable state for the next batch.

This gives a D:G update ratio of **1:1**. There is no label smoothing, additional regularization, or mixed-precision training in these runs.

3.4 Sampling, logging, and checkpointing

Each epoch saves G/D losses, D's mean probabilities on real and generated images, the number of samples and updates, elapsed time, a fixed-noise sample grid, and a checkpoint. Epoch metrics are weighted by batch size, so the last 96-image batch does not receive the same total weight as a full 128-image batch. The logged D probabilities come from the **D-stage forward passes before its optimizer step**.

The 64 training-display noise vectors use a separate generator with seed **2026**. Sampling switches G to evaluation mode, disables gradient recording, and restores its previous mode afterward. This prevents display sampling from updating BatchNorm statistics or consuming the training-noise random stream.

Each checkpoint contains G/D weights, BatchNorm buffers, both Adam states, configuration, epoch, metrics, and random states. The main comparison consistently uses the **epoch-20 checkpoint**.

4. Experimental settings

The formal experiments use batch size **128**, latent dimension **100**, Adam betas **(0.5, 0.999)**, weight decay **0**, and training seed **42**. Each run creates fresh models, optimizers, and a shuffled loader. E2 changes only the common G/D learning rate; E3 changes only generator BatchNorm.

E0 uses a smaller subset and two epochs to exercise the complete training pipeline. It is excluded from the formal quality ranking. E1-E3 each complete 469 batches per epoch and **9,380 optimizer updates per network** over 20 epochs. The tables below are read from the recorded configuration and environment files.

In [ ]:
```
show_table(report, 'experiments')
show_table(report, 'environment')
```

5. Training process

5.1 Loss curves and discriminator responses

Figure 2 plots all 20 raw epoch means for each formal run. Initial fluctuations are followed by smaller changes in the epoch-level losses and discriminator probabilities. The baseline G loss remains near 1 during much of the later training, while its D loss is also near 1. E3 starts with a larger G loss and approaches a more stable range later.

In [ ]:
```
show_figure(report, 'outputs/comparison/seed_42/training_comparison.png')
show_table(report, 'losses')
```

Figure 2. Raw sample-weighted G/D losses and D-stage mean probabilities.

5.2 Generated images as training progresses

A 64-image grid was saved after every epoch. Figure 3 arranges the baseline checkpoints into the requested **5-row by 4-column** progression: rows correspond to epochs **1, 5, 10, 15, and 20**, and each column reuses the same noise vector. The four vectors are the first four from the fixed seed-2026 display set.

The first row contains loosely organized strokes. By epoch 5, the examples have clearer digit-like shapes. Later stages refine the strokes while largely retaining the identities associated with these inputs. This shows a learning trajectory;

In [ ]:
```
show_figure(report, 'outputs/comparison/seed_42/baseline_progress.png', width=54
```

Figure 3. Baseline generation progress. Rows are training stages and columns are fixed noise inputs.

6. Generated examples

Each epoch-20 generator produces 200 images from the same independent evaluation noise set, with seed **2027**. Figure 4 displays exactly the **first 64** images from each set.

A common noise input controls sampling across configurations, but it does not impose a digit identity. A particular position can therefore depict different digits

in E1 and E3 without that difference being an error.

In [ ]:
```
show_figure(report, 'outputs/comparison/seed_42/final_samples.png')
```

Figure 4. Epoch-20 examples for E1, E2, and E3, using the same evaluation inputs and fixed display range. No image was replaced or filtered based on quality.

Checkpoint-based generation demonstration

In [ ]:
```
#Reloads the saved E1 checkpoint and runs G on the first eight saved evaluation
show_checkpoint_demo(report, experiment='E1', count=8)
```

Figure 5. Eight examples regenerated by the report from the baseline checkpoint.

7. Human evaluation and sample diversity

The 200 images from each configuration were presented on annotation sheets with anonymous set identifiers A-C. I assigned a digit label 0-9 or `unreadable` by myself; Recognizability is the fraction assigned a digit label. Class coverage counts the distinct recognizable digit labels. Unreadable images remain in the 200-image denominator.

The baseline has 179 recognizable samples, compared with 168 for E2 and 164 for E3. Under this fixed 20-epoch budget, lowering the learning rate reduces the observed recognizability by **5.5 percentage points**, and removing generator BatchNorm reduces it by **7.5 percentage points**.

I did not label instances where the cause could not be identified, and since there are quite a few images that are difficult to distinguish, manual labeling may result in some errors.

In [ ]:
```
show_table(report, 'quality')
show_figure(report, 'outputs/comparison/seed_42/class_distribution.png')
```

Figure 6. Human label counts across all 200 images per run.

Digit 1 occurs only **4, 4, and 3** times in E1-E3, whereas digit 9 occurs **38, 37, and 46** times. The generated distributions are therefore not balanced.

Human-labeled failure examples

Figure 7 shows the first four `unreadable` examples per experiment in ascending sample-ID order. This deterministic selection documents human-identified failures without replacing them with more favorable outputs.

In [ ]:
```
show_failure_examples(report)
```

Figure 7. Human-labeled unreadable samples.

8. Training and inference time

Training time covers the epoch loop, including batch updates, per-epoch sample generation, checkpoint writes, metric writes, and progress output. It excludes data/model preparation, the epoch-0 preview, and final aggregate plotting. GPU work is synchronized at timing boundaries. The table reports the original measured times from the completed runs.

Inference time measures G generating a batch of **1,000 images** in evaluation mode under `no_grad`. Noise is allocated on the device before timing, using seed 2028.

In [ ]:
```
show_table(report, 'runtime')
```

Amortized time per image is the batch mean divided by 1,000.

9. Discussion, limitations, and conclusion

Experimental Approach: First, establish a moderately-sized, easily interpretable convolutional GAN baseline; then, conduct single-factor experiments to examine the effects of the learning rate and the generator's BatchNorm. Using an Unconditional GAN with a convolutional architecture

Data: The experiment used 60,000 training images from the original MNIST dataset, retaining the 1×28×28 image dimensions. No binarization or data augmentation was performed. The batch size was 128; the validation set consisted of 469 batches per round, with the final batch containing 96 images. The testing set consisted of 47 batches per round, with the final batch containing 112 images.

Training Setup: In the formal experiment, each group was trained for 20 rounds, with G and D updated 9,380 times each; the weights from the 20th round were ultimately used for all groups, and the top 64 images from the evaluation set are shown.

Network: The network architecture is described in Chapter 3 above. The sequence of operations for each batch is as follows:

1. Update D: Generate a fake image, apply `detach()` to the fake image, compute the loss between the true and generated images, and perform one Adam update for D.
2. Update G: Resample the noise, freeze D's parameters, compute the loss using the new fake image against target 1, and perform one Adam update for G.
3. Restore D's parameters, then next batch.

Some conclusions and summary:

- Figure 3: Shows the output of rounds 1, 5, 10, 15, and 20 for the same four noise patterns. As can be seen, the early results consist of blurry, loose strokes; by round 5, the digit shapes are already fairly clear; and subsequent rounds primarily focus on improving clarity.
- In the manual evaluation, the baseline E1 performed best, with recognition rates of 89.5%, 84%, and 82% for E1, E2, and E3, respectively.
- Small convolutional GANs are already capable of generating a variety of recognizable digits from noise.
- Reducing the joint learning rate and removing the generator's BatchNorm both decreased the recognizability rate within the fixed budget of this experiment.
- All three groups still exhibited blurriness, mixed character shapes, and skewed class distributions; neither generation quality nor diversity has been fully resolved.

These results support two qualified conclusions:

1. Learning rate experiments: After halving the shared learning rate, the number of distinguishable samples decreased by 11, a drop of 5.5 percentage points, and no improvement in quality was observed within the 20-round budget.
2. BatchNorm ablation: After removing BatchNorm from G, the number of distinguishable samples decreased by 15, a drop of 7.5 percentage points, which supports retaining BatchNorm in this baseline.
3. Class imbalance in manual annotation may be due to a skewed distribution of classes learned by the generator; at the same time, manual interpretation of some blurry images may also affect the statistics. In the actual training set, there aren't particularly many instances of class 9. The statistics for the original training set show class balance and are unrelated to this issue.

Possible reasons for suboptimal results: using only a single training seed, evaluating only 200 samples per configuration, relying on only one human annotator, a lack of quantitative measures of intra-class replication, and unstable runtime conditions.

10. Code organization and reproducibility

File	Responsibility and connection
prepare_data.py	Acquires raw MNIST, normalizes images, saves the complete tensor and fixed debug indices, and exports data statistics
mnist_data.py	Loads processed tensors and supplies image-only, shuffled batches to training
models.py	Implements the G/D modules described in Section 3
train.py	Defines E0-E3 configurations, runs alternating Adam updates, and writes metrics, fixed samples, and checkpoints

File	Responsibility and connection
evaluate.py	Reloads epoch-20 weights, samples common evaluation noise, times inference, and aggregates human labels
report_utils.py	Loads recorded results and renders the short tables, image displays, and checkpoint demonstration used here
debug.ipynb	Maintains the project's data, model, training, and evaluation debugging workflow

The report's tables read comparison/seed_42/summary.json, with the human labels in annotations.csv. Each formal run retains its configuration, raw epoch metrics, 20 checkpoints, fixed-noise samples, and 30 raw inference measurements. comparison.csv and class_counts.csv provide machine-readable summaries.

To open this report, use the project directory and select any compatible Python kernel. Install the packages listed in requirements.txt in that environment. Reading the saved outputs requires only this executed notebook. Rerunning the short cells requires the accompanying Python modules and saved `outputs/` artifacts, including the E1 epoch-20 checkpoint for the demonstration. The recorded training/benchmark environment describes the original runs; it is not a requirement for the current kernel.

Training reproduction is separate from report execution. For example, the following command starts a new E1 run in a distinct directory; replace the experiment and output directory for E2 or E3:

```
python train.py --experiment E1 --device auto --output-dir
outputs/E1/seed_42_repeat
```

The report does not execute that command. Its generation demonstration is a forward pass from existing weights, and its runtime table preserves the original benchmark measurements.